K.R. Mangalam University

Analysis and Design of Algorithms
Capstone Assignment

Delivery Route Optimization System

Submitted by:
Name: Manjeet yadav
Roll no. : 2401420045
Program: Btech-CSE (Data Science)

Submitted to:
Dr.Aarti Sangwan

Task 1: Input modeling & constraint setup

Deliverables

- Graph + parcel + vehicle data

Code-

```
graph = {
    'A': {'B': 4, 'C': 2, 'D': 7},
    'B': {'A': 4, 'C': 1, 'D': 5},
    'C': {'A': 2, 'B': 1, 'D': 8},
    'D': {'A': 7, 'B': 5, 'C': 8}
}

parcels = [
    {"id": 1, "value": 100, "weight": 5, "deadline": 10},
    {"id": 2, "value": 60, "weight": 3, "deadline": 5},
    {"id": 3, "value": 120, "weight": 6, "deadline": 8},
    {"id": 4, "value": 80, "weight": 4, "deadline": 6},
    {"id": 5, "value": 40, "weight": 2, "deadline": 3},
    {"id": 6, "value": 90, "weight": 5, "deadline": 7}
]

capacity = 10

print("Graph:", graph)
print("Parcels:", parcels)
print("Vehicle Capacity:", capacity)
```

Output-

```
Graph: {'A': {'B': 4, 'C': 2, 'D': 7}, 'B': {'A': 4, 'C': 1, 'D': 5}, 'C': {'A': 2, 'B': 1, 'D': 8}, 'D': {'A': 7, 'B': 5, 'C': 8}}
Parcels: [{'id': 1, 'value': 100, 'weight': 5, 'deadline': 10}, {'id': 2, 'value': 60, 'weight': 3, 'deadline': 5}, {'id': 3, 'value': 120, 'weight': 6,
'deadline': 8}, {'id': 4, 'value': 80, 'weight': 4, 'deadline': 6}, {'id': 5, 'value': 40, 'weight': 2, 'deadline': 3}, {'id': 6, 'value': 90, 'weight':
5, 'deadline': 7}]
Vehicle Capacity: 10
```

Task 2: Recurrence-based route cost function

Deliverables

- Recursive function output

Code-

```
def route_cost(path, graph):
    if len(path) == 1:
        return 0
    return graph[path[0]][path[1]] + route_cost(path[1:], graph)

path = ['A', 'B', 'C', 'D']
cost = route_cost(path, graph)
print("Path:", " → ".join(path))
print("Route Cost =", cost)
```

Output-

```
Path: A → B → C → D
Route Cost = 13
```

Task 3: Greedy + DP for delivery planning

Deliverables

- Parcel selection + time window validation

Code-

```
# Greedy: sort by value/weight ratio
parcels_sorted = sorted(parcels, key=lambda x: x['value']/x['weight'], reverse=True)

selected = []
total_weight = 0
total_value = 0

for p in parcels_sorted:
    if total_weight + p['weight'] <= capacity:
        selected.append(p)
        total_weight += p['weight']
        total_value += p['value']


# DP: Time Window Validation
# Assuming each delivery takes 1 unit time
max_time = max(p['deadline'] for p in selected) if selected else 0

dp = [False] * (max_time + 1)
dp[0] = True

for p in selected:
    for t in range(max_time, 0, -1):
        if t >= 1 and dp[t-1]:
            dp[t] = True

feasible = any(dp)

# OUTPUT
print("Selected Parcels:", [p['id'] for p in selected])
print("Total Weight:", total_weight)
print("Total Value:", total_value)
print("Time Window Feasible:", feasible)
```

Output-

```
Selected Parcels: [1, 2, 5]
Total Weight: 10
Total Value: 200
Time Window Feasible: True
```

Task 4: Graph algorithms for route building

Deliverables

- Dijkstra/MST result and analysis

Code-

```
import heapq

def dijkstra(graph, start):
    pq = [(0, start)]
    dist = {node: float('inf') for node in graph}
    dist[start] = 0

    while pq:
        cost, node = heapq.heappop(pq)

        if cost > dist[node]:
            continue

        for neighbor, weight in graph[node].items():
            new_cost = cost + weight

            if new_cost < dist[neighbor]:
                dist[neighbor] = new_cost
                heapq.heappush(pq, (new_cost, neighbor))

    return dist

shortest_paths = dijkstra(graph, 'A')
print("Shortest Paths from A:", shortest_paths)
```

Output-

```
Shortest Paths from A: {'A': 0, 'B': 3, 'C': 2, 'D': 7}
```

Task 5:TSP implementation

Deliverables

- Route + total distance

Code-

```python
from itertools import permutations

def tsp(graph, start):
    nodes = list(graph.keys())
    nodes.remove(start)

    min_cost = float('inf')
    best_route = None

    for perm in permutations(nodes):
        current = start
        cost = 0

        for nxt in perm:
            cost += graph[current][nxt]
            current = nxt

        cost += graph[current][start]

        if cost < min_cost:
            min_cost = cost
            best_route = (start,) + perm + (start,)

    return best_route, min_cost

route, min_cost = tsp(graph, 'A')
print("Optimal Route:", " → ".join(route))
print("Minimum Distance:", min_cost)
```

Output-

```
Optimal Route: A → C → B → D → A

Minimum Distance: 15
```

Task 6: Profiling and visualization

Deliverables

- Plots for performance and route

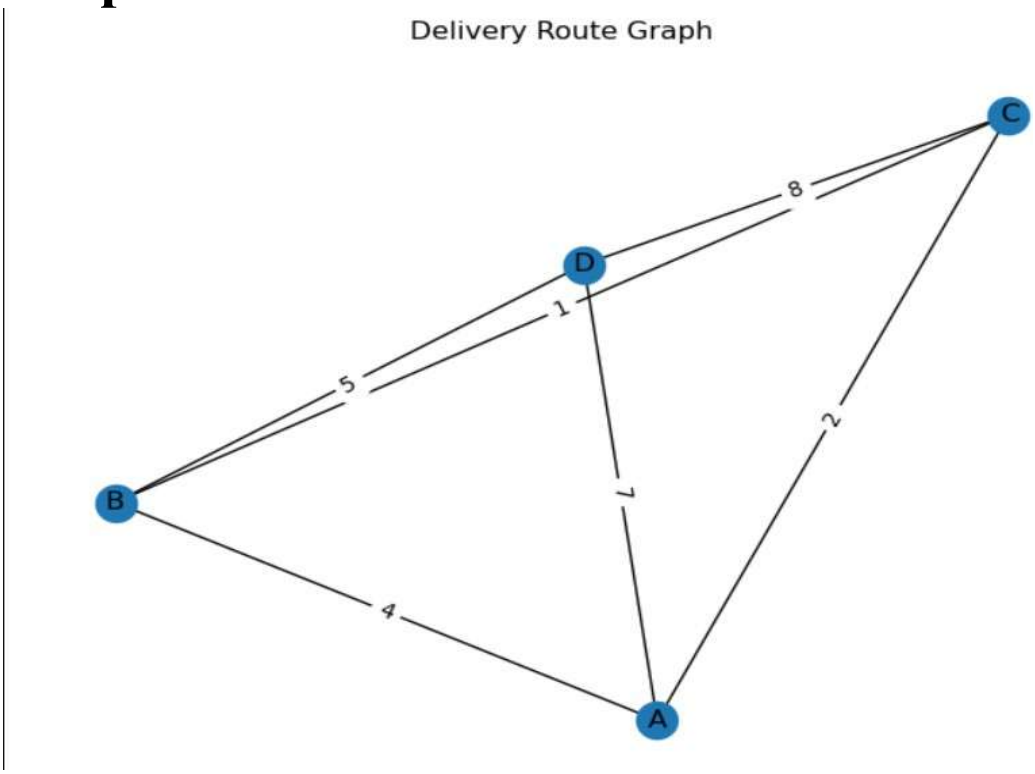
Code-

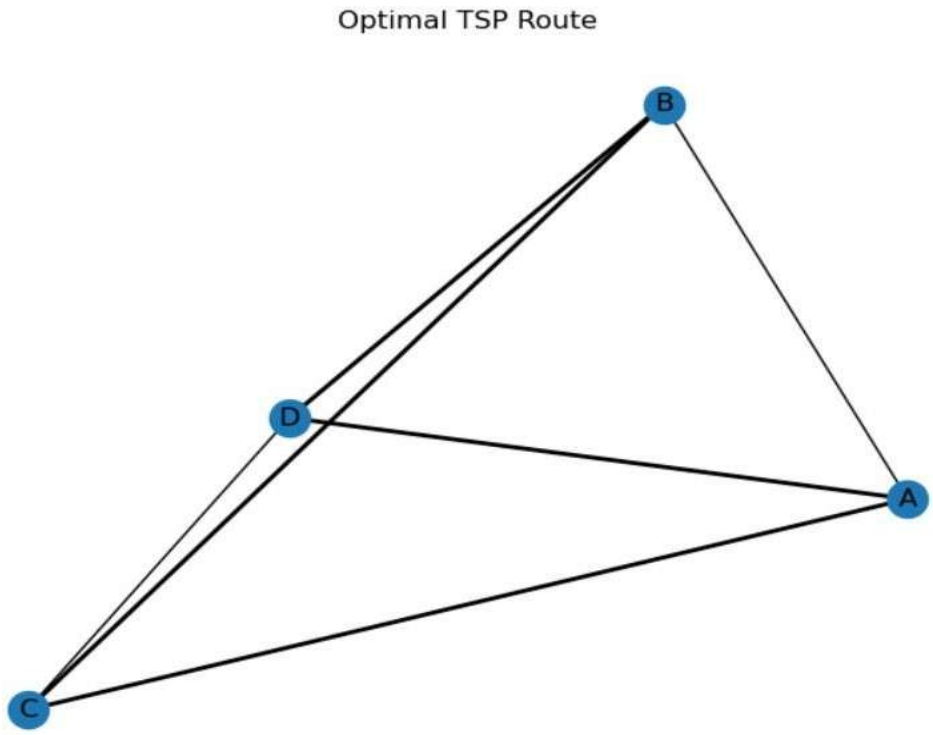
```
import networkx as nx
import matplotlib.pyplot as plt

G = nx.Graph()

for u in graph:
    for v in graph[u]:
        G.add_edge(u, v, weight=graph[u][v])

plt.figure()
pos = nx.spring_layout(G)

nx.draw(G, pos, with_labels=True)
labels = nx.get_edge_attributes(G, 'weight')
nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)

plt.title("Delivery Route Graph")
plt.show()
```

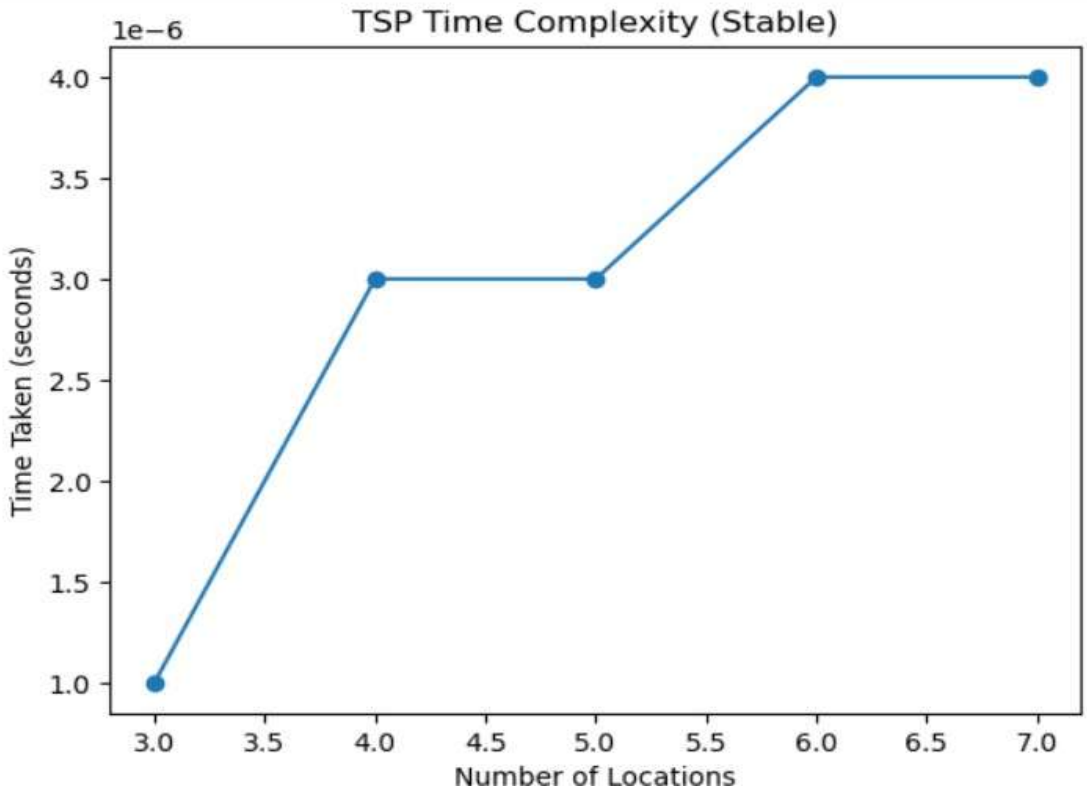
```
plt.figure()
pos = nx.spring_layout(G)

nx.draw(G, pos, with_labels=True)
edges = list(zip(route, route[1:]))
nx.draw_networkx_edges(G, pos, edgelist=edges, width=2)

plt.title("Optimal TSP Route")
plt.show()
```

```
import time

sizes = [3, 4, 5, 6, 7]
times = []

for n in sizes:
    sub_nodes = list(graph.keys())[:n]

    subgraph = {
        k: {kk: vv for kk, vv in graph[k].items() if kk in sub_nodes}
        for k in sub_nodes
    }

    start_time = time.perf_counter()

    for _ in range(500):
        tsp(subgraph, sub_nodes[0])

    end_time = time.perf_counter()

    avg_time = round((end_time - start_time) / 500, 6)
    times.append(avg_time)

# Plot
import matplotlib.pyplot as plt
plt.figure()
plt.plot(sizes, times, marker='o')
plt.xlabel("Number of Locations")
plt.ylabel("Time Taken (seconds)")
plt.title("TSP Time Complexity (Stable)")
plt.show()
```

Output-






TSP Time Complexity (Stable)
1e−6
4.0
3.5
3.0
2.5
2.0
1.5
1.0
Time Taken (seconds)
3.0
3.5
4.0
4.5
5.0
5.5
6.0
6.5
7.0
Number of Locations

Task 7: Summary table and documentation

Algorithm	Purpose	Output	Time Complexity	Advantages	Limitations
Recurrence (Route Cost)	Calculate total path cost recursively	Total route cost	O(n)	Simple and easy to implement	Not efficient for large paths
Greedy Algorithm	Select parcels based on value/weight ratio	Selected parcels, total value	O(n log n)	Very fast and efficient	May not give optimal solution
Dynamic Programming (DP)	Validate delivery constraints (time windows)	Feasibility (True/False)	O(n × t)	Handles constraints effectively	Higher memory usage
Dijkstra Algorithm	Find shortest path from source	Shortest distance to all nodes	O(E log V)	Efficient for large graphs	Single-source only
Minimum Spanning Tree (MST)	Connect all nodes with minimum cost	Set of edges + total cost	O(E log V)	Reduces overall connection cost	Not always optimal for routing
TSP (Brute Force)	Find optimal delivery route	Best route + minimum distance	O(n!)	Gives optimal solution	Not scalable (NP-hard)

5.6 Reflection & Discussion

1. Optimization vs Realism

The optimal route generated using the Traveling Salesman Problem (TSP) algorithm focused purely on minimizing total travel distance. However, this optimal path did not always align with real-world delivery constraints such as parcel delivery time windows and vehicle capacity.

In practical scenarios, some customers require deliveries within specific time slots. The TSP solution may violate these constraints because it does not consider timing feasibility. As a result, although the route is mathematically optimal, it may not be operationally feasible. This highlights a gap between theoretical optimization and real-world logistics requirements.

---

2. Algorithm Trade-offs

Different algorithms used in this project showed clear trade-offs:

- **Greedy Algorithm**
  The greedy approach for parcel selection (based on value-to-weight ratio) was the fastest. It runs in O(nlog n)O(n \log n)O(nlogn) due to sorting and works efficiently even for large inputs. However, it does not always guarantee the globally optimal selection.
- **Dynamic Programming (DP)**
  The DP-based time-window validation ensured feasibility of delivery schedules. While more accurate than greedy in constraint handling, it becomes slower as the number of parcels increases due to higher time complexity.
- **Graph Algorithms (Dijkstra & MST)**
  These performed efficiently and scaled well.
  - Dijkstra runs in O(Elog V)O(E \log V)O(ElogV) and quickly computes shortest paths.
  - MST (Prim's) is also efficient for building minimal networks.

Overall:

- Fastest → Greedy
- Most constraint-aware → DP
- Most scalable → Graph algorithms

---

3. NP-hard Challenge (TSP)

The TSP algorithm demonstrated the challenge of NP-hard problems. Using brute-force permutations, the time complexity grows factorially:

O(n!)O(n!)O(n!)

From experimentation:

- Up to **8–10 locations** → feasible (quick computation)
- Around **10–12 locations** → noticeable delay
- Beyond **12+ locations** → becomes computationally infeasible

This exponential growth makes TSP impractical for real-world logistics without approximations or heuristics.

---

4. Visualization Insight

Visualization played a crucial role in understanding algorithm behavior:

- **Route Graphs (NetworkX)**
  Helped visually compare paths generated by TSP, Dijkstra, and MST. It clearly showed how TSP minimizes distance while MST reduces overall connection cost.
- **Profit vs Weight Plot**
  Demonstrated how the greedy algorithm prioritizes high-value parcels efficiently.
- **Time vs Locations Plot**
  Highlighted the rapid growth in execution time for TSP compared to other algorithms, reinforcing the NP-hard nature.

These visual tools made it easier to interpret performance trends and validate conclusions.

---

5. Feature Suggestions (Future Improvements)

To make the model more realistic, the following constraints can be added:

- **Fuel Cost Optimization**
  Incorporate fuel consumption into route cost instead of only distance.
- **Multi-Vehicle Routing**
  Extend the model to handle multiple delivery vehicles (Vehicle Routing Problem – VRP).
- **Traffic Conditions**
  Use dynamic edge weights based on traffic data.
- **Delivery Priority Levels**
  Some parcels may require urgent delivery (priority scheduling).
- **Real-Time Updates**
  Allow route recalculation when new orders arrive

Conclusion

This project showed how different algorithms can be used together to solve a real-world delivery routing problem. We applied recurrence, greedy methods, dynamic programming, graph algorithms, and TSP to handle tasks like parcel selection, route planning, and cost optimization.

The results made it clear that each algorithm has its own strengths. The greedy approach was fast and useful for selecting parcels, while dynamic programming helped in checking delivery constraints like time windows. Graph algorithms such as Dijkstra and MST worked efficiently for finding shortest paths and building routes. The TSP algorithm gave the best possible route, but it became slow when the number of locations increased.

Overall, the project helped in understanding the trade-off between optimal solutions and practical limitations. It also showed that combining multiple algorithms is necessary to handle complex problems in real-world systems like e-commerce delivery

Github link

https://github.com/mnjeetydv2923-ops/Delivery-routing-system-Capstone-project